Architecture decisions

Each entry records one technical fork: the decision, the context, the alternatives, why, and what it costs. Superseding a decision means adding a new entry, not editing this one.

ADR-1: Read captures with dpkt, not PyShark

- **Decision.** Use dpkt 1.9.8 for PCAP and PCAPNG.
- **Context.** The tool reads captures offline on the target machine. PyShark is a wrapper over `tshark`, a separate binary.
- **Alternatives.** PyShark; scapy.
- **Why.** dpkt reads both formats in pure Python with no external process, so the dependency set stays installable with `pip` alone. PyShark would put `tshark` on the critical path of a machine we do not control. scapy is heavier and slower for up to 250k packets and pulls a larger dependency tree.
- **Consequences.** We own link-layer and IPv4/IPv6 handling (Ethernet, loopback and BSD cooked are understood; the rest are reported as diagnostics). pcapng support is only as good as dpkt's; the corpus and fixtures exercise it.

ADR-2: Build the corpus as PCAPNG

- **Decision.** The reproducible corpus and the defect fixtures are PCAPNG.
- **Context.** The case allows PCAP and PCAPNG. The fixtures are generated by `scripts/generate_corpus.py` and `scripts/generate_fixture.py`.
- **Alternatives.** Classic PCAP.
- **Why.** PCAPNG carries per-interface metadata and a packet comment field, and it is what Wireshark writes by default, so the corpus matches what a real capture looks like. The parser reads both, so choosing one costs nothing.
- **Consequences.** Any pcapng-only quirk in dpkt affects the whole corpus; the fixtures double as a parser regression test. A classic PCAP is read in tests to keep both paths alive.

ADR-3: Store annotations in SQLite, not a JSON file

- **Decision.** A project keeps `annotations.sqlite` for researcher notes.
- **Context.** Notes are appended during a session and must survive a move of the project directory; the manifest already points at the store.
- **Alternatives.** A JSON or JSONL file next to the manifest.
- **Why.** Appending to SQLite is atomic and does not rewrite a growing file; a lookup by offset or session key is a query, not a scan. The notes are not part of the normalized contract, so a richer store costs nothing on the wire.
- **Consequences.** The project directory holds one binary file that must be copied as-is; the export archive carries it unchanged. A reader without SQLite cannot read the notes, but every contract artifact stays JSON.

ADR-4: Desktop UI in PySide6, not a web interface

- **Decision.** The interface is a PySide6 desktop window.
- **Context.** The case asks for a local desktop application and gives a Linux x86-64 target.
- **Alternatives.** A local web app (browser front end, Python back end); Tkinter.
- **Why.** A desktop window matches the case, runs offline with no server or port, and PySide6 ships Qt in a wheel so no system Qt is needed. A web front end would add a server, a port, and a browser dependency for no gain on a single-machine tool. Tkinter cannot render the dense table and hex views the case needs without heavy custom work.
- **Consequences.** A GUI test needs a display; the suite uses the offscreen Qt platform and skips when the Qt runtime is absent. Packaging a Qt app is larger than a pure-Python CLI.

ADR-5: Frame by testing the length field's three readings, not assuming one

- **Decision.** The framing step tries three readings of the length field and keeps only one that frames every stream with no leftover bytes.
- **Context.** The length field's meaning is not written down. It could count the payload, the payload plus the length field, or the whole message.
- **Alternatives.** Assume the common "payload only" reading and move on.
- **Why.** The two wrong readings do not fail loudly, they produce a plausible first message and then chaos. Testing all three against the real bytes on all ten streams (see `docs/REFERENCE_INVESTIGATION.md`) shows one reading frames everything and the others do not, which is evidence, not a guess.
- **Consequences.** Framing must be able to reject a candidate, so a candidate is judged by "frames every stream cleanly" rather than "produced something".

ADR-6: Interpretations are declarative rules, not scripts

- **Decision.** An interpretation is a versioned JSON or YAML rule applied by an engine.
- **Context.** A rule describes a frame and its fields, and must be reused on new data, versioned, and shared.
- **Alternatives.** A small Python script per interpretation.
- **Why.** A rule can be validated against a schema, versioned, diffed, applied by the CLI, and shipped in a project without executing anything. A script is flexible but cannot be versioned or validated the same way, and applying a stranger's script to your data is a code-execution risk.
- **Consequences.** The rule schema is a ceiling: fields are flat, and a nested or variable-length structure needs a schema change. That limit is recorded in `docs/ROADMAP.md`.

ADR-7: Keep the engine in process, do not export to Kaitai

- **Decision.** Apply rules with the in-process engine; do not generate a Kaitai Struct spec as the parsing path.
- **Context.** Kaitai Struct is a declarative format that can express binary layouts and generate parsers.
- **Alternatives.** Generate Kaitai from the rule and parse with it.
- **Why.** The engine keeps the provenance link: every field result carries the stream range it came from, so a click in the window returns to the source packet. A Kaitai export would either lose that

link or need its own mapping back, and it adds a code-generation toolchain to a tool that must run offline from `pip`.

- **Consequences.** We maintain the field readers ourselves. In exchange, the result format is ours and the provenance survives end to end.

ADR-8: Default to in memory, offer streaming for large captures

- **Decision.** The default path holds the capture in memory; a streaming mode releases closed sessions as it walks.
- **Context.** The reference profile is up to 100 MiB and 250k packets. Peak memory scales with the capture because every packet is kept for provenance.
- **Alternatives.** Stream by default; memory-map the file.
- **Why.** In-memory keeps provenance trivial and the code clear, and at the reference profile it peaks at 350 MiB against a 4 GiB budget. Streaming is available where memory is tighter; both produce byte-identical output, which the tests assert.
- **Consequences.** A capture several times larger needs streaming or a session-by-session parse. The measured trade-off is in `docs/BENCHMARK.md`, and the next step is recorded in `docs/ROADMAP.md`.

ADR-9: Ship a CLI pipeline as well as the window

- **Decision.** Every stage has a command-line entry, and a pipeline command runs capture to report in one step.
- **Context.** Requirement R6 asks for reuse of an interpretation outside the original session.
- **Alternatives.** Build the GUI only.
- **Why.** The CLI makes the tool scriptable and reproducible, which is what a reviewer or a later run needs, and it runs headless. The window is for exploration; the CLI is for repetition. The same engine backs both, so they cannot drift.
- **Consequences.** Two surfaces to keep consistent. The tests drive the CLI and the window against the same corpus to catch drift.

ADR-10: Score field readings algorithmically, do not call a model

- **Decision.** Name a `hypothesis` field by scoring candidate readings with support and contradiction over the corpus.
- **Context.** Fields marked `hypothesis: true` have a guessed meaning. The case forbids presenting a guess as a fact.
- **Alternatives.** Ask a language model to name fields from examples.
- **Why.** The scoring is deterministic and offline, so the same bytes give the same score every time, which the reproducibility requirement needs. A model would make the result depend on an external service and a prompt, and would not be checkable. `docs/REFERENCE_INVESTIGATION.md` shows the scoring picking `low_cardinality` for `command` and `target` while the declared `constant` reading loses, so the method can contradict a guess.
- **Consequences.** The method only ranks readings the researcher supplies; it does not invent them. That gap is the first item in `docs/ROADMAP.md`.

ADR-11: Gaps are markers, never zero-filled

- **Decision.** A missing segment is a `gap` diagnostic with an offset and a length; the stream is not padded with zeros.
- **Context.** A capture can be incomplete, and a rule must not read invented bytes as data.

- **Alternatives.** Fill the hole with zeros so offsets line up.
- **Why.** Zero-filling would present invented bytes as observation, which the case forbids, and a field over the hole would read a plausible value that is not in the capture. A message over a gap is classified `incomplete`.
- **Consequences.** A consumer of the stream must handle gaps explicitly. In exchange, an offset either maps to a source packet or is known to be missing.

ADR-12: Conflicting overlaps stay ambiguous

- **Decision.** When two packets overlap with different bytes, keep the first bytes and record the contradiction as `ambiguity`, holding both versions.
- **Context.** Retransmissions can carry altered bytes; a real stack resends or the capture is inconsistent.
- **Alternatives.** Prefer the later packet; prefer the earlier; pick by heuristic.
- **Why.** Any silent choice would present one version as the stream and hide the conflict. The case requires the ambiguity to be visible. Keeping the first bytes is a stated rule, and the diagnostic carries the alternative.
- **Consequences.** A message over an ambiguous range is classified `ambiguous`, not `matched`. The window shows both versions.

ADR-13: Interface language via Qt translators, not a home-grown map

- **Decision.** Keep every UI string in English in the source, wrap it in `tr()`, and switch language with a `QTranslator` over a compiled Qt catalogue.
- **Context.** The window needs a Russian and an English interface, chosen at run time and remembered across restarts.
- **Alternatives.** A dictionary of `{source: translation}` looked up by hand at every assignment; a third-party i18n library.
- **Why.** `tr()` is idiomatic Qt, `pyside6-lupdate` harvests the strings so none is missed by hand, and `QTranslator` is the mechanism Qt rebuilds widgets with. A hand-rolled map would need a lookup call at every `setText` and would drift from the code as strings change. The dictionary approach also cannot relabel a widget once it is built.
- **Consequences.** Widgets that build their own labels expose a `retranslate_ui()`; the window calls it on every registered view when the language changes, so a switch is live and needs no restart. Strings the window assembles itself, such as the rule-status line and the provenance card, are kept as a renderer and rebuilt on a switch rather than stored as finished text, which would otherwise freeze in the language it was built in. Both languages ship a compiled catalogue: English is the source language, so its catalogue mirrors the source strings one for one, and it is committed alongside the Russian one so neither language falls back to a half-translated interface. The compiled `.qm` is committed so the installed binary works without a Qt toolchain. The language choice is a `QSettings` value under the `madrigal/protocol-lab` scope.